Contents

ĐẶC TẢ YÊU CẦU PHẦN MỀM 1
1. Giới thiệu 2
1.1 Mục đích 2
1.2 Phạm vi sản phẩm (Kiến trúc 3 tầng) 2
2. Mô tả tổng quan 2
2.1 Bối cảnh sản phẩm 2
2.2 Chức năng sản phẩm 2
2.3 Sơ đồ Use Case tổng quan (Use Case Diagram) 3
3. Mô hình cơ sở dữ liệu (Database Schema) 4
3.1 Bảng ParkingSlots (Quản lý vị trí đỗ) 4
3.2 Bảng ParkingLogs (Lịch sử ra vào bãi) 4
4. Đặc tả tầng nghiệp vụ (BLL) & Thuật toán cốt lõi 5
4.1 Thuật toán Check-in tự động tìm ô trống 5
4.1.2 Sơ đồ Trình tự Check-in (Sequence Diagram) 6
4.2 Thuật toán Check-out giải phóng ô đỗ 6
4.2.2 Sơ đồ Trình tự Check-out (Sequence Diagram) 7
4.3 Luồng thực thi ứng dụng (Activity Diagram / Swimlane) 8

ĐẶC TẢ YÊU CẦU PHẦN MỀM

Hệ thống Quản lý Bãi đỗ xe thông minh (Smart Parking System)

Mục	**Nội dung**
Môn / Đồ án	Kỹ thuật Phần mềm (KTPM 20261)
Sinh viên thực hiện	1. Phạm Trung Kiên — MSSV: 20250776P 2. Trần Minh Khoa — MSSV: 20250777P
Giảng viên hướng dẫn	Vũ Song Tùng

1. Giới thiệu

1.1 Mục đích

Tài liệu này mô tả chi tiết các yêu cầu chức năng, phi chức năng và kiến trúc kỹ thuật đối với phần mềm **Quản lý Bãi đỗ xe thông minh (Smart Parking System)**. Tài liệu đóng vai trò là căn cứ kĩ thuật cho việc phát triển, kiểm thử và bảo vệ đồ án trước hội đồng đánh giá tại Đại học Bách Khoa Hà Nội.

1.2 Phạm vi sản phẩm (Kiến trúc 3 tầng)

Hệ thống được thiết kế theo mô hình kiến trúc 3 tầng (3-tier architecture) nhằm phân định rõ ràng trách nhiệm giữa giao diện người dùng, logic nghiệp vụ và lưu trữ dữ liệu:

- **Tầng 1 — Giao diện / Thiết bị (Presentation / Client Tier):** Giao diện ứng dụng (Console App / WinForms / WPF) hỗ trợ nhân viên vận hành thực hiện nhập biển số xe vào/ra bãi và quan sát trạng thái trực quan của các ô đỗ.
- **Tầng 2 — Nghiệp vụ (Business Logic Layer - BLL):** Xử lý các quy tắc nghiệp vụ quản lý bãi đỗ (tự động tìm ô trống khả dụng, kiểm tra trạng thái IsOccupied, tính tiền gửi xe dựa trên thời gian thực, điều phối nhật ký vào/ra).
- **Tầng 3 — Dữ liệu (Data Access Layer - DAL / Server):** Quản trị cơ sở dữ liệu quan hệ SQL Server (SmartParkingDB) với các bảng chính (ParkingSlots, ParkingLogs), tương tác thông qua Entity Framework Core.

2. Mô tả tổng quan

2.1 Bối cảnh sản phẩm

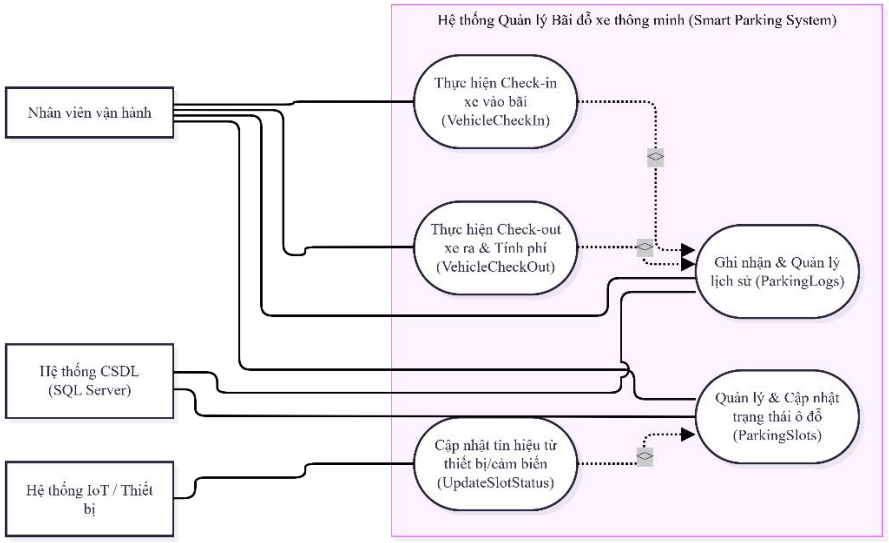
Bãi đỗ xe hiện đại yêu cầu mức độ tự động hóa cao trong quy trình kiểm soát xe vào, xe ra, quản lý danh mục ô đỗ theo thời gian thực. Hệ thống giúp giảm thiểu sự can thiệp thủ công, tránh tình trạng quá tải và hạn chế tối đa sai sót ghi chép dữ liệu.

2.2 Chức năng sản phẩm

- **Quản lý vị trí đỗ (ParkingSlots):** Kiểm soát danh sách các vị trí đỗ cố định (ví dụ: A-01, A-02, A-03, A-04) và cập nhật trạng thái khả dụng (IsOccupied = 0 hoặc 1).
- **Check-in xe vào:** Tự động tìm kiếm ô đỗ trống đầu tiên
- theo thứ tự ưu tiên, cập nhật trạng thái ô đỗ sang đã sử dụng (IsOccupied = 1), đồng thời khởi tạo bản ghi lịch sử xe vào bảng ParkingLogs.

- **Check-out xe ra:** Ghi nhận mốc thời gian xe ra, tính toán cước phí gửi xe, đồng thời **giải phóng ô đỗ** (IsOccupied = 0) để phục vụ các lượt xe tiếp theo.

2.3 Sơ đồ Use Case tổng quan (Use Case Diagram)



Mục đích: Cung cấp bức tranh tổng quan về hệ thống, xác định rõ hệ thống làm được những gì và ai là người (hoặc hệ thống khác) sử dụng các chức năng đó.

Các thành phần chính:

- **Tác nhân (Actors):**
    - *Nhân viên vận hành:* Tương tác trực tiếp để thực hiện nghiệp vụ cốt lõi như Check-in, Check-out, quản lý ô đỗ và theo dõi lịch sử.
    - *Hệ thống IoT / Thiết bị:* Tự động gửi tín hiệu cập nhật trạng thái cảm biến ô đỗ (UpdateSlotStatus).
    - *Hệ thống CSDL (SQL Server):* Lưu trữ và cung cấp dữ liệu cho các chức năng.
- **Mối quan hệ <<include>> (Bắt buộc):** Các chức năng nghiệp vụ như VehicleCheckIn hay VehicleCheckOut khi thực thi bắt buộc phải đi kèm với việc cập nhật trạng thái ô đỗ (ParkingSlots) và lưu nhật ký giao dịch (ParkingLogs).

3. Mô hình cơ sở dữ liệu (Database Schema)

3.1 Bảng ParkingSlots (Quản lý vị trí đỗ)

Bảng lưu trữ thông tin cố định của các ô đỗ trong bãi:

Trường (Column)	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa nghiệp vụ**
SlotId	INT	PK, IDENTITY	Khóa chính, định danh vị trí đỗ (1, 2, 3, 4...)
SlotName	NVARCHAR(50)	NOT NULL	Tên hiển thị ô đỗ (Ví dụ: A-01, A-02...)
IsOccupied	BIT	DEFAULT 0	Trạng thái ô đỗ (0: Trống, 1: Đã có xe đỗ)
LastUpdated	DATETIME	DEFAULT GETDATE()	Thời điểm cập nhật trạng thái gần nhất

3.2 Bảng ParkingLogs (Lịch sử ra vào bãi)

Bảng lưu trữ nhật ký giao dịch gửi xe:

Trường (Column)	**Kiểu dữ liệu**	**Ràng buộc**	**Ý nghĩa nghiệp vụ**
LogId	INT	PK, IDENTITY	Khóa chính lịch sử giao dịch
SlotId	INT	FOREIGN KEY	Liên kết tới ParkingSlots(SlotId)
LicensePlate	NVARCHAR(20)	NOT NULL	Biển số xe gửi
TimeIn	DATETIME	DEFAULT GETDATE()	Thời điểm xe vào bãi

Trường (Column)	Kiểu dữ liệu	Ràng buộc	Ý nghĩa nghiệp vụ
TimeOut	DATETIME	NULL	Thời điểm xe ra khỏi bãi
Status	NVARCHAR(20)	DEFAULT 'IN'	Trạng thái lượt gửi (IN: Đang đỗ, OUT: Đã thanh toán và ra)
TotalFee	DECIMAL(18,2)	NULL	Tổng tiền gửi xe tính toán khi check-out

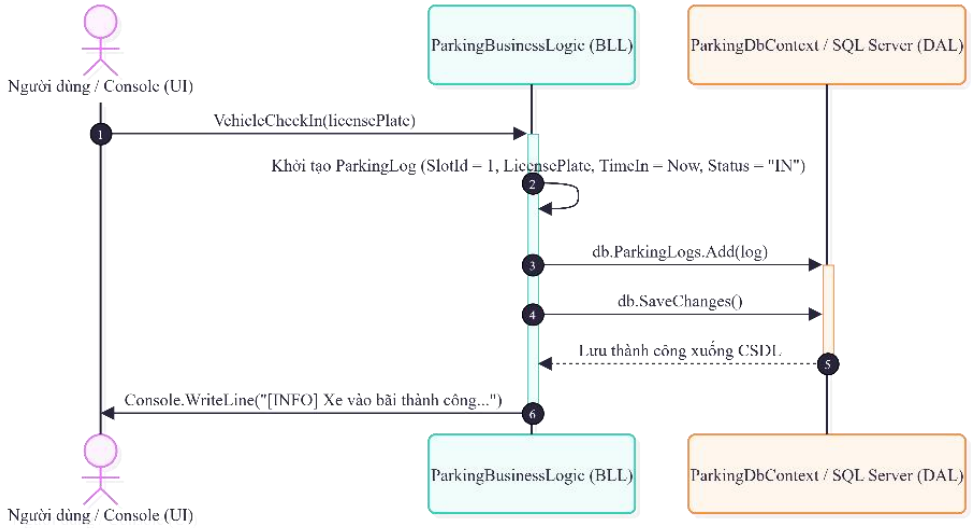
4. Đặc tả tầng nghiệp vụ (BLL) & Thuật toán cốt lõi

4.1 Thuật toán Check-in tự động tìm ô trống

Để đảm bảo tính linh hoạt và chính xác, tầng BLL chủ động thực hiện truy vấn điều phối ô đỗ:

1. Truy vấn tìm ô đỗ đầu tiên có IsOccupied == false trong bảng ParkingSlots.
2. **Nếu không tìm thấy:** Trả về thông báo "Bãi đỗ xe đã đầy".
3. **Nếu tìm thấy:**
    - Lấy giá trị SlotId tương ứng của ô đỗ trống (Ví dụ: SlotId = 1 cho A-01).
    - Cập nhật IsOccupied = true cho vị trí này.
    - Khởi tạo bản ghi ParkingLog mới liên kết chính xác với SlotId vừa chọn và ghi nhận TimeIn = DateTime.Now.

4.1.2 Sơ đồ Trình tự Check-in (Sequence Diagram)



Mục đích: Mô tả chi tiết trình tự các thông điệp (messages) và luồng gọi hàm giữa các tầng (UI → BLL → DAL) khi có xe xin vào bãi.

Các bước diễn ra:

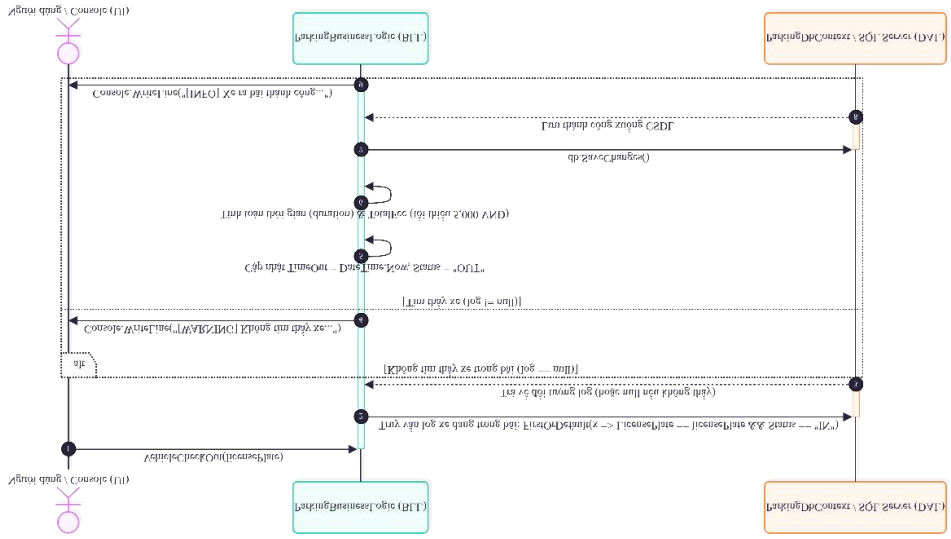
1. **Người dùng / Giao diện (UI)** gọi phương thức VehicleCheckIn(licensePlate) truyền vào biển số xe.
2. Tầng Nghiệp vụ (**BLL**) khởi tạo một đối tượng ParkingLog mới với thông tin: SlotId = 1, biển số xe, thời gian vào TimeIn = DateTime.Now, và gán trạng thái Status = "IN".
3. BLL gọi lệnh thêm vào DbSet (db.ParkingLogs.Add(log)) và lưu thay đổi xuống cơ sở dữ liệu (db.SaveChanges()).
4. Tầng Dữ liệu (**SQL Server / DAL**) thực thi câu lệnh SQL và phản hồi lại kết quả thành công.
5. BLL in thông báo thành công ra màn hình Console ([INFO] Xe vào bãi thành công...) để kết thúc luồng.

4.2 Thuật toán Check-out giải phóng ô đỗ

1. Tìm kiếm lượt gửi xe đang active trong ParkingLogs theo LicensePlate và Status == "IN".

2. Cập nhật TimeOut = DateTime.Now, đổi Status = "OUT", và tính toán TotalFee theo công thức quy định.
3. Lấy SlotId tương ứng từ bản ghi log, thực hiện cập nhật lại IsOccupied = false trong bảng ParkingSlots để giải phóng vị trí đỗ.

4.2.2 Sơ đồ Trình tự Check-out (Sequence Diagram)



Mục đích: Mô tả chi tiết luồng xử lý khi xe rời bãi, bao gồm việc kiểm tra tính hợp lệ, tính tiền cước và giải phóng chỗ đỗ.

Các bước diễn ra:

1. **UI** gọi phương thức VehicleCheckOut(licensePlate).
2. **BLL** gửi câu lệnh truy vấn tới **DAL** để tìm kiếm xe đang đỗ trong bãi (LicensePlate == ... và Status == "IN").
3. **Cấu trúc rẽ nhánh (alt):**

- *Nhánh 1 (Không tìm thấy xe):* Trả về log == null, hệ thống in cảnh báo [WARNING] Không tìm thấy xe đang trong bãi!.
- *Nhánh 2 (Tìm thấy xe):* Cập nhật thời gian ra TimeOut = DateTime.Now, đổi trạng thái thành Status = "OUT", tính toán thời gian gửi và tiền cước phí (TotalFee tối thiểu 5.000 VNĐ). Sau đó gọi db.SaveChanges() để lưu dữ liệu xuống SQL Server và thông báo thành công ra màn hình.

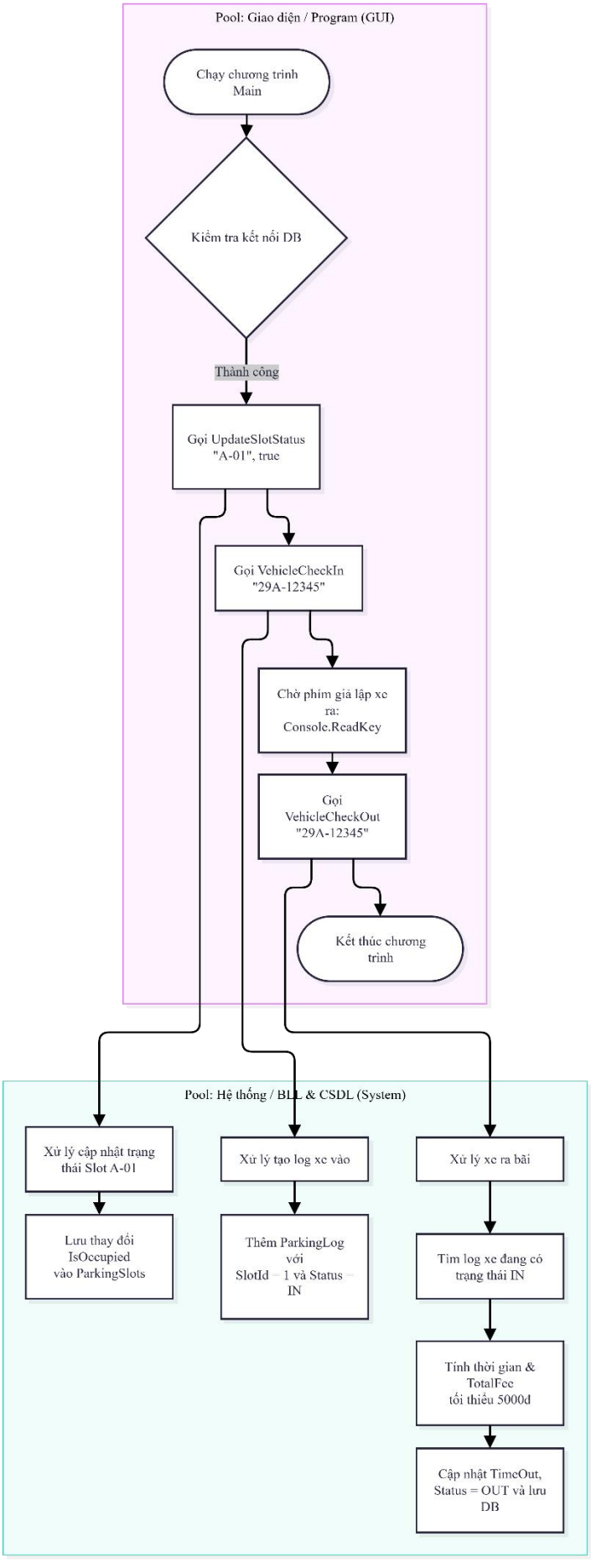
4.3 Luồng thực thi ứng dụng (Activity Diagram / Swimlane)

Sơ đồ thể hiện luồng hoạt động tổng thể theo phân tầng giữa Giao diện (GUI/Program) và Tầng hệ thống (BLL & CSDL):

Mục đích: Phân định rõ ràng trách nhiệm xử lý (Separation of Concerns) giữa phía **Giao diện người dùng (GUI/Client)** và phía **Hệ thống lõi (System/Backend & CSDL)**.

Nội dung chia thành 2 Pool:

- **Pool GUI (Giao diện / Program):** Chịu trách nhiệm khởi chạy chương trình (Main), kiểm tra kết nối cơ sở dữ liệu ban đầu, gọi các hàm giả lập trạng thái ô đỗ (UpdateSlotStatus), thực hiện lệnh Check-in, chờ phím nhấn giả lập thời gian thực (Console.ReadKey) và gọi lệnh Check-out kết thúc.
- **Pool System (Hệ thống / BLL & CSDL):** Lắng nghe và xử lý các yêu cầu từ GUI thực hiện ở tầng ngầm: cập nhật trạng thái IsOccupied vào bảng ParkingSlots, thêm bản ghi IN vào bảng ParkingLogs, thực hiện truy vấn tìm xe, tính toán cước phí và cập nhật trạng thái OUT lưu trực tiếp xuống cơ sở dữ liệu SQL Server.


Pool: Giao diện / Program (GUI)
Chạy chương trình Main
Kiểm tra kết nối DB
Thành công
Gọi UpdateSlotStatus "A-01", true
Gọi VehicleCheckIn "29A-12345"
Chờ phím giả lập xe ra: Console.ReadKey
Gọi VehicleCheckOut "29A-12345"
Kết thúc chương trình
Pool: Hệ thống / BLL & CSDL (System)
Xử lý cập nhật trạng thái Slot A-01
Lưu thay đổi IsOccupied vào ParkingSlots
Xử lý tạo log xe vào
Thêm ParkingLog với SlotId = 1 và Status = IN
Xử lý xe ra bãi
Tìm log xe đang có trạng thái IN
Tính thời gian & TotalFee tối thiểu 5000đ
Cập nhật TimeOut, Status = OUT và lưu DB

Mô tả các mối quan hệ chính:

- **Nhân viên vận hành:** Tương tác trực tiếp với các chức năng Check-in, Check-out, tra cứu lịch sử ParkingLogs và xem danh sách vị trí đỗ ParkingSlots.
- **Hệ thống IoT / Thiết bị:** Đóng vai trò cập nhật tự động tín hiệu nhận diện/trạng thái ô đỗ vào hàm UpdateSlotStatus.
- **Mối quan hệ <<include>>:** Luồng VehicleCheckIn và VehicleCheckOut bắt buộc bao gồm việc cập nhật lại trạng thái ô đỗ trong bảng ParkingSlots và lưu nhật ký giao dịch vào bảng ParkingLogs.